

Performance Analysis of Adaptive Runge-Kutta Methods in Region of Interest *

Alarico Campetelli, Georg Hackenberg
Institut für Informatik
Technische Universität München
Garching bei München, 85748, Germany
{campetel, hackenbe}@in.tum.de

Abstract: The combination of embedded systems with physical components, termed Cyber-Physical Systems (CPSs), poses important challenges in their design, modelling, simulation and verification. Particularly challenging are the interaction of discrete software elaborations with analogues time evolutions and integration of artefacts from different engineering disciplines in CPSs. The increasing importance of the software components in CPSs determines needs for a integrated model-based tool supporting in all design phases. We presented within the framework of FOCUS modelling theory a logical representation tailored for software-intensive CPSs: hybrid i/o state machines [Cam13]. We introduced also algorithms for the simulation in discrete time time steps of such state machines in a modelling support tool [Cam13]. We believe that simulation and validation of software-intensive CPSs in early development phases permits an agiler and more efficient design process. In this paper we analyse a preliminary implementation of our sampling algorithms with a focus on performance and scalability aspects. We compare the results with dynamic sampling algorithms used in MATLAB Simulink/Stateflow¹ that is de facto an industrial standard for the design of embedded systems and CPSs.

Keywords: Hybrid systems, embedded systems, model-based development, sampling, hybrid systems simulation

1 Introduction

Safety-critical CPSs combine mechanical components, electronic parts with discrete software units. This determines the necessity of a suitable design that unite continuous and discrete behaviour. FOCUS [BDD⁺92] is a modelling theory for the formal specification of distributed, discrete-event embedded systems. FOCUS components are defined in a hierarchical and interconnected net, each with a typed i/o interface. Systems modelled with FOCUS are at a logical representation level that contains the behaviour and the interface

*This work was partially supported by the DFG (German Research Foundation) under the Priority Programme SPP1593: Design for Future - Managed Software Evolution. The responsibility for this article lies with the authors.

¹<http://www.mathworks.com>

of the systems, but abstracting their technical implementation. The FOCUS theory is supported and implemented in the modelling tool AutoFOCUS 3² [HF11]. AutoFOCUS 3 is designed for the development of reactive, software-intensive, embedded systems. It is implemented on top of the Eclipse platform³. In this tool, systems are modelled by software architectures composed of components with executable behaviour descriptions (e.g., automata with input and output). We introduced in [Cam13] an extension of the FOCUS modelling theory that permits to define components with a support for continuous time elaborations. Our i/o FOCUS hybrid state machines are inspired by the well-known hybrid automaton formalism [Hen96]. In this formalism the continuous variables elaborate their continuous values with differential equations.

Since i/o FOCUS state machines utilise differential equations associated to continuous variables, numerical algorithms are required to solve them. In the literature a variety of approaches exist for the solution of differential equations. Probably the most well-known approach for solving ordinary differential equations over the time domain is the Runge-Kutta method [Alb96]. The general idea behind the Runge-Kutta method is to approximate the differential equation at discrete time points only. These time points can be defined at constant or adaptive distance. There are many adaptive Runge-Kutta methods [Feh70, DP80, BS89, CK90]. During simulation adaptive Runge-Kutta methods typically adjust the time step size depending on the approximation error. The approximation error is estimated typically using different orders of approximation for differential equations.

Problem Current adaptive Runge-Kutta methods are based on the approximation error at every discrete time point. Consequently, these methods try to approximate the differential equations as exact as possible across the entire simulation time interval. However, focusing on the entire simulation time interval can become quite time consuming. In particular, the simulation of large systems of differential equations might suffer from the approximation error-based adoption policy. Therefore, the simulation of such systems might become infeasible even taking into account today's computing power. To overcome this situation we have been working on adoption policies, which rather work on regions of interest than on error approximation. The general idea behind region of interest adaptive Runge-Kutta methods is to require high approximation quality only for specific regions of the differential equation, while accepting lower approximation quality for all other regions.

Contribution In this article we propose two region-of-interest adaptive Runge-Kutta methods for simulation of ordinary differential equations and provide an initial evaluation of their performance. In contrast to existing approximation error-based adaptive Runge-Kutta methods, our algorithms increase or decrease the time step size depending on the values that are reached by the continuous variables in the first approach, and on their slope in the second approach. These adoption policies have been selected due to the observation, that particular variable and slope ranges provide good representatives for regions of interest. In order to analyse the performance of our region of interest adaptive Runge-Kutta methods, we further introduce a prototypical implementation in MATLAB. Finally,

²<http://autofocus.in.tum.de>

³<http://www.eclipse.org>

we compare our implementation with widely used approximation error-based adaptive Runge-Kutta methods in the de facto standard modelling tool for CPSs, namely MATLAB Simulink/Stateflow (MSS).

Outline This work is structured as follows: in the next section we introduce the theoretical foundations of our dynamic sampling approach and we explain the applied approximation method for differential equations. In the third section, we describe the characteristics of our dynamic sampling algorithms and their prototypical implementation in MATLAB. The evaluation with a case study is presented in Section 4. Finally, in the last section we give summary, concluding remarks and future work.

2 Foundations

Considering the discrete character of FOCUS models and the continuous dynamic of hybrid automata, we defined *hybrid components* in [Cam13]. A hybrid component communicates through input and output channels over continuous hybrid streams functions. The internal behaviour of each hybrid component is implemented by a special version of hybrid automaton [Hen96], called i/o FOCUS hybrid state machine. We report here the definitions of these state machines and the architecture of our sampling approach.

Definition 1 (I/O Hybrid State Machine) *An i/o hybrid state machine is a tuple $H = (\Sigma, Var, Init, I, O, Dom, E, f, G, R)$*

- *A state space $\Sigma = (Q \times V)$ where Q is a set of discrete states $Q = \{q_1, q_2, \dots\}$ and V a set of continuous states $V \subseteq (M_1^*)^{\mathbb{R}^+} \times \dots \times (M_l^*)^{\mathbb{R}^+} \times (M_{l+1})^{\mathbb{R}^+} \times \dots \times (M_n)^{\mathbb{R}^+}$, to each element of V corresponds a hybrid stream, that is associated to a variable in the variable set Var . Each M_i set is the set of messages that are reached by the corresponding hybrid stream function.*
- *A set of initial states $Init \subseteq \Sigma$.*
- *A set of input $I \subseteq V$ and output $O \subseteq V$ state space, respectively for input and output channels. Channels are derived from the i/o interface of the state machine. $O_d \subseteq O$ and $O_c \subseteq O$, with $O_c \cap O_d = \emptyset$ are respectively the output state space associated to continuous and discrete stream functions.*
- *A set of internal continuous $Int \subseteq V$ state space and internal discrete $Int_d \subseteq V$ state space both with no channels associated, with $I \cap O \cap Int \cap Int_d = \emptyset$. We denote with $W = Int \cup O_c$ the set of internal and output state spaces.*
- *A domain function $Dom : Q \rightarrow \mathcal{P}(V)$.*
- *A set of edges $E \subseteq Q \times Q$ that represent the discrete state transition.*
- *A vector field function $f : Q \times V \rightarrow W$.*
- *A guard condition function $G : E \rightarrow \mathcal{P}(V)$.*

- A reset map function $R : E \times V \rightarrow \mathcal{P}(W \cup O_d)$ that represent with the vector field the continuous dynamics.

Definition 2 (Sampled Hybrid Component) A hybrid sampled i/o hybrid component corresponding to the i/o hybrid component H is the following map $v_H : \mathcal{S}_{N_1} \times \dots \times \mathcal{S}_{N_n} \rightarrow \mathcal{S}_{M_1} \times \dots \times \mathcal{S}_{M_m}$

Definition 3 (Hybrid Controller) A hybrid controller is a map $\mathcal{C} : \mathcal{S}_{M_1} \times \dots \times \mathcal{S}_{M_m} \rightarrow \mathcal{S}_{N_1} \times \dots \times \mathcal{S}_{N_n}$

where the N_i and M_i are respectively the input and output message sets for i/o channels of H . The set of outputs that arrive to the controller are received at sampling intervals, so the controller is activated at a sampling rate $\Delta > 0$ and provides the corresponding input for the next sampled period.

2.1 Approximation of Differential Equations

We provide a solution based on numerical mathematics for the execution of the dynamic sampling algorithms presented in the next section. This solution is necessary to discretise and approximate the differential equations defined in each discrete state of the hybrid i/o state machines. There are first order differential equations associated with internal or output continuous variables, according to the vector field function $f : Q \times V \rightarrow W$ (Definition 1). In linear hybrid i/o machines, differential equations define the behaviour of continuous variables $w \in W \cup O_d$ in each discrete state, resulting in a system of n differential equations with n real variables. There are many classes of numerical methods for ODEs, a well known classification is in the following two classes: the *one-step methods* that use one initial value at each step and the *multistep methods* that use more values to compute the solution [HNW93]. One-step solutions calculate the point y_{i+1} by using only the point y_i as initial value and do not use any previously computed solution points. Instead the multistep methods use the $k + 1$ previously computed solution values $y_i, y_{i-1}, \dots, y_{i-k}$ to calculate the next solution point y_{i+1} . Generally, multistep methods are slower but produce more accurate solutions than one-step methods.

We presented hybrid i/o state machines as a general modelling paradigm for CPSs. Considering the heterogeneity of the modelled systems is it difficult to estimate the complexity of typical differential equations, so we focus on one-step methods. Computer controlled systems utilize mainly these three different discretization methods: Euler, Zero-Order-Hold, and Runge-Kutta method [SBB⁺02, OWN96]. The Euler method elaborates a correct approximation of systems if the sample period is very small, anyway others methods generally provide better accuracy of the approximation. The Zero-Order-Hold method has been used in industrial applications, especially in situations where the state information is discontinuous or intermittent. It uses the most recent information until new values are received, so it is a method more suitable for systems with time-delayed inputs. The Runge-Kutta method is a higher order approach, compared to the other two. We think that this

approach works best with our i/o linear hybrid state machines. In fact, the limitations of the Euler method and the characteristics of the Zero-Order method are not well suited for our models. The Runge-Kutta method is a classical fourth-order method, which guarantees a perfect balance between accuracy and complexity. Moreover, this method has many scientific applications and can work very effectively as ODE integrator, especially when combined with adaptive stepsize algorithms [PTVF92].

Given a differential equation $\dot{w} = f(q, v)$ for a continuous variable $w \in W$, where without loss of generality $w(t) = v_j \in V$ in $v(t) = (v_0, \dots, v_n)$, and a period defined by the function $\delta : \mathbb{N} \rightarrow \mathbb{R}_+$, then the value $w(\delta(i+1)) = f(q(\delta(i+1)), v(\delta(i+1))) = w_{i+1}$ is calculated with the classical Runge-Kutta method as follows:

$$\begin{aligned} w_i &= f(q(\delta(i)), v(\delta(i))) \\ h &= \delta(i+1) - \delta(i) \end{aligned}$$

$$\begin{aligned} W_1 &= w_i \\ W_2 &= w_i + h/2 f(q(\delta(i)), v^j(\delta(i), W_1)) \\ W_3 &= w_i + h/2 f(q(\delta(i) + h/2), v^j(\delta(i) + h/2, W_2)) \\ W_4 &= w_i + h f(q(\delta(i) + h/2), v^j(\delta(i) + h/2, W_3)) \end{aligned}$$

$$w_{i+1} = w_i + h/6 (w_i + 2f(q(\delta(i) + h/2), v^j(\delta(i), W_2)) + 2f(q(\delta(i) + h/2), v^j(\delta(i), W_3)) + f(q(\delta(i+1)), v^j(\delta(i+1), W_4)))$$

where $v^j(t, x) = (v_0, v_1, \dots, v_j - 1, x, v_{j+1}, \dots, v_n)$ with $v(t) = (v_0, \dots, v_n)$ and V has cardinality n . This sequence of steps can be executed by a computer and there are already many implementations, as for instance in [PTVF92], where the algorithm has been implemented in C programming language. It requires more computational time than the Euler's method, but provides also more accurate results. An advantage of Runge-Kutta is that it can work effectively also with long sampling periods. The classical Runge-Kutta method presented above is an approximation for the single variable w , but can be generalised to find an approximate solution of a system of differential equations.

3 Dynamic Sampling Algorithms

We introduce in this section two approaches for the sampling of i/o hybrid state machine [Cam13]. The first approach is based on the calculation of the slope for the functions defined by differential equations and a correspondent acceptance value for it. The sampling is considered stable if the slope is smaller or equal than an acceptance value.

Definition 4 (Period variation based on the slope) *Given a hybrid component H , a set of acceptance values L_i and a stabilization times S , with $0 \leq i < |W|$. We define the dynamic variation of the period as follows: we calculate the slope from the second step at time $t = 2d_0$ and for the following steps. The evolution of the system is guided from the differential equation over the variable in $w \in W$, that is $\dot{w} = f(q, v)$, where (q, v) is*

the actual state and $\delta(n-1)$ is the actual period of the component H for the n -th step of elaboration.

At elaboration step n for each variable $w \in W$ we use the previous value of w at the computation step $n-1$ to compute the slope:

$$\text{slope}(w, n) = \frac{w(n) - w(n-1)}{\delta(n-1)}$$

We update the period for the next computation step only if the slope is greater than the acceptance value L_w ; or until this elaboration step for a time interval equal or greater than the stabilization time S the period was constant and smaller than the maximum value d_{max} . This calculation is done for all variables $w \in W$. At the end if at least one variable need to change the period its actual value is halved, until a minimal value d_{min} . The mapping δ is inductively defined: for an initial state (q_0, v_0) the mapping is set for the first step of elaboration to an initial value d_0 :

$$\delta(0) = d_0$$

$$\delta(i+1) = \begin{cases} \delta(i)/2 \text{ (} d_{min} \text{ if } \delta(i)/2 < d_{min}) & \text{if } \exists w \in W \text{ with } |\text{slope}(w, i)| > L_w \\ \delta(i) * 2 \text{ (} d_{max} \text{ if } \delta(i) * 2 > d_{max}) & \text{if } (\forall w \in W |\text{slope}(w, i)| \leq L_w \wedge \\ & |\text{slope}(w, i-1)| \leq L_w \wedge \dots \wedge \\ & |\text{slope}(w, i-k)| \leq L_w) \wedge (\delta(i) + \\ & \delta(i-1) + \dots + \delta(i-k) \geq S) \wedge \text{with } k \leq i \\ \delta(i) & \text{otherwise} \end{cases} \quad (1)$$

In the second approach, if the value of a variable is in a critical interval and the actual period is greater than the acceptance period associated to the interval, then the period is set to this value. The sampling period for the next computation step is calculated for each output or internal variable, and finally will be chosen the minimum period.

Definition 5 (Period variation based on intervals) Given a hybrid component H , a set of critical intervals K , a set of period times P , with $|K| = |P| = |W|$ and $k \in K, p \in P, k, p \in \mathbb{R}_+$, we build the dynamic variation of the period as follows: for the initial states (q_0, v_0) the period is set to the maximum value d_0 , at the second step of elaboration (at time $t = 2d_0$) the dynamic choice of the period starts. The evolution of the system is guided from the differential equation over the variable in $w \in W$, that is $\dot{w} = f(q, v)$, where (q, v) is the actual state.

The modification of the actual period is made analysing the actual value of the variables in W , which is if a variable w is in a critical interval K_w , and then the corresponding P_w acceptance period is the necessary value of w for the period. It is finally chosen the minimum acceptance period between the variables that are in a critical interval. The period map δ is inductively over the elaboration steps as follow determined:

$$\begin{aligned}
\delta(0) &= d_0 \\
w_{\delta_{i+1}} &= \begin{cases} P_j & \text{if } w(i) \in K_j, \text{ with } 0 \leq j < |W| \\ \delta(i) & \text{otherwise} \end{cases} \\
\delta(i+1) &= \min\{w_{\delta_{i+1}} \mid w \in W\}
\end{aligned} \tag{2}$$

3.1 Implementation of the Sampling Algorithms in MATLAB

Differential equations in which all the derivatives are respect to only one independent variable are called ordinary differential equations (ODEs). ODEs describe usually the evolution of the state of continuous time systems. In our i/o state machine we use ODEs for the variable in the set W . There are different ODE solvers to approximate differential equations. We decided for a fourth order Runge-Kutta because it guarantees a perfect balance between accuracy and complexity. Moreover, this method has many scientific applications and can work very effectively as ODE integrator, especially when combined with dynamic sampling algorithms. We implemented prototypically our sampling algorithms in MATLAB (version R2014a) and we selected some models for a preliminary analysis of our implementations. Simulink/Stateflow is a model-based data flow graphical tool integrated in the MATLAB environment, which can drive the elaboration with MATLAB or be scripted from it. Simulink/Stateflow support modelling and simulation of dynamic systems and is an industrial standard in control theory and signal sampling.

There are many dynamic and static sampling algorithms in the simulation environment of Simulink/Stateflow. We analysed widely used ODE solvers in the following case study. The selected ODE solvers are similar to our sampling solutions, in fact some of them are also Runge-Kutta based methods. We used the toolbox MSS for two purposes: (1) a comparison between modelling and verification of FOCUS hybrid component in MSS and in AutoFOCUS 3; and (2) an initial test of our sampling solutions against the most used dynamic sampling algorithms in MSS. We consider the dynamic sampling solver *ode45* integrated in MSS, which provides fourth and fifth order formulas and is based on an algorithm of Dormand and Prince.

We programmed in MATLAB our sampling algorithms as ODEs solver functions. The resulting two MATLAB functions have different parameters and can be executed over continuous functions, as made by the MATLAB function *ode45*. We call *ode4slope* the implementation of the algorithm based on the slope; and *ode4interval* the implementation of the algorithm based on critical intervals. Unfortunately, the implemented algorithms in MATLAB cannot be utilized as ODE solvers in the Simulink/Stateflow environment. We cannot therefore test our sampling solutions using Simulink models. Anyway, we can test them on differential equations defined in the MATLAB language.

4 Evaluation

We select a derived form of the Mathieu’s differential equation for a preliminary evaluation of our solutions. It is a system of two linear differential equations representing an oscillating trigonometric function. Mathieu functions are special functions used in mathematics for treating a variety of problems [Zwi97]. The MATLAB code of the differential equation we used for testing our algorithms is shown in Listing 1.

Listing 1: MATLAB Code for Mathieu’s differential equation.

```
function r = test(t,x)
% delta
d = 0.1045;
% epsilon
e = 0.0048685;
% result (2-dimensional)
r = [x(2); (-d - e * cos(t)) * x(1) - 0.7 * d * abs(x(1))];
end
```

We simulated the derived form of Mathieu’s differential equation, which was mentioned in the MSS community website MATLAB Central⁴, in a time interval of one hundred thousand seconds, with the following ODE solvers: *ode45*, *ode4slope*, and *ode4interval*. We configured *ode45* with default parameters and our solutions with an initial period of 0.6, minimal period of 0.2 and maximal period of 2.0 seconds. We set for the *ode4slope* algorithm the following parameters: an acceptance value of 0.55 for all variables and a stabilization time of 1.0 second. Meanwhile, we set for the *ode4interval* algorithm the critical interval $[-1.0, 1.0]$ for the variable $x(1)$ and $[-0.5, 0.5]$ for the variable $x(2)$ with an acceptance period of 0.5 seconds for both variables. These parameters are chosen to have a similar number of simulation steps of the algorithm *ode45*. The results of the simulation with the mentioned parameter values are summarized in Table 1. The values reached by the variables $x(1)$ (in blue) and $x(2)$ (in green) in the simulation are graphical represented in Figures 1, 2 and 3.

Sampling Algorithm	Simulation Steps	%	Computation Time	%
<i>ode45</i>	157.089	100	4.2665 seconds	100
<i>ode4slope</i>	196.768	125	4.0702 seconds	95
<i>ode4interval</i>	182.531	116	3.7980 seconds	89

Table 1: Simulation results in terms of absolute/relative simulation steps and computation time.

As shown in Table 1, our algorithms *ode4slope* and *ode4interval* calculate more simulation steps for the same simulation time interval (i.e. 16 - 25% more compared to *ode45*), while requiring less overall computation time (i.e. 5 - 11% less compared to the *ode45*). This performance improvement is due to the fact that we do not rely on estimating the approximation error. As stated before, to us the approximation error is less important because we

⁴<http://www.mathworks.com/matlabcentral>

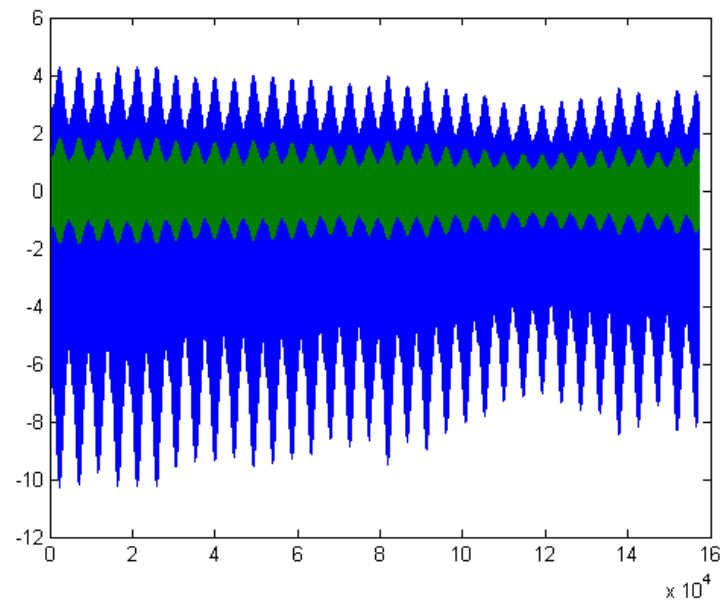


Figure 1: Simulation of the function *test* with the algorithm *ode45*.

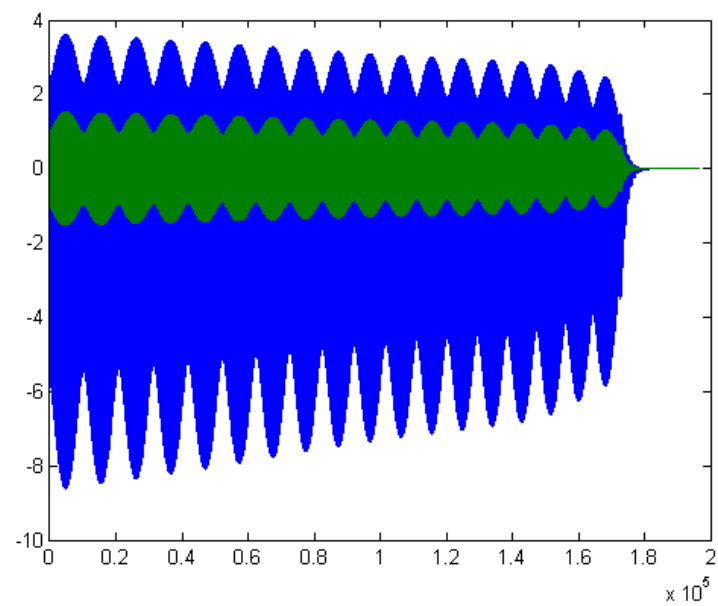


Figure 2: Simulation of the function *test* with the algorithm *ode4slope*.

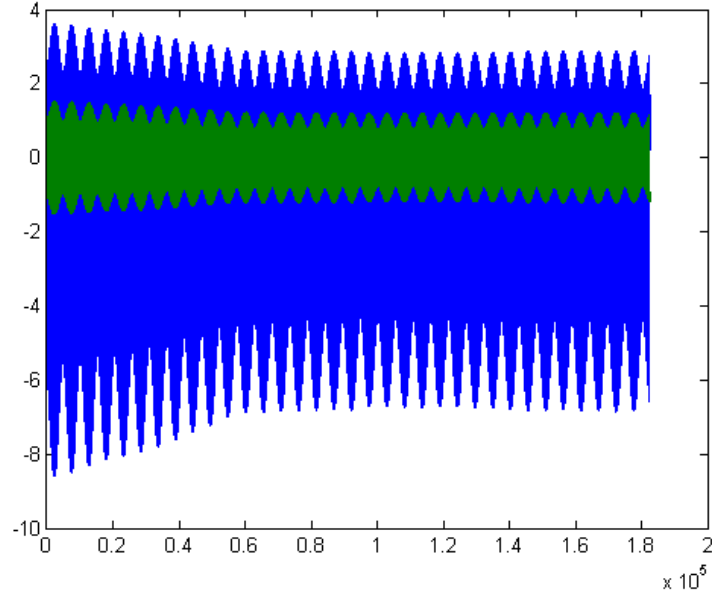


Figure 3: Simulation of the function *test* with the algorithm *ode4interval*.

aim at supporting different needs. Our solutions have the following advantages: Firstly, the overall computational performance is better and generally more stable and predictable performance can be guaranteed with respect to existing approaches. Unfortunately, to the best of our knowledge no public benchmark sets are available for comparing adaptive sampling algorithms with respect to performance. The initial performance evaluation with the presented example, which due to its simplicity can be considered a good representative benchmark, shows promising results for the scalability of our solution. In particular, scalability is interesting for systems with a large number of elements, where a simulation within reasonable amount of time is more important than the precision of the simulation itself. For instance, simulation performance might be more important than simulation precision in early development phases of systems with a very large number of components. Secondly, an advantage of our approaches is the direct setting of period lengths in desired variable value intervals and the setting of acceptance values for the slope to determine the desired change of the period. Consequently, it is possible to initialize simulations with a desired precision. For instance, a possible scenario is the validation of a system, defining a shorter period when error states are reached. Parameters for the sampling algorithms could be directly derived by system requirements. Thirdly, we study an approach to formally verify hybrid i/o state machines with the model checker HyCOMP⁵. We check formal assertions over continuous variables modelled by differential equations. Formal assertions, as invariants, could be verified in an automatic or assisted way to guide the determination

⁵<http://es-static.fbk.eu/tools/hycomp>

of parameters, which are necessary for our sampling algorithms. The verification process of HyCOMP consider an infinite time horizon. Therefore, our solutions can use information about the behaviour of the continuous variables in an infinite time interval, which is an advantage with respect to existing sampling algorithms as for instance in the MSS environment. In fact, they usually consider the variable behaviour in finite time intervals.

5 Conclusion and Future Work

We studied the execution in continuous time of variables guided by differential equations and simulated in a modelling tool as AutoFOCUS 3. We implemented prototypically our sampling solutions using MATLAB language and we executed preliminary tests. We compared our solutions with sampling algorithms in the industrial standard for model-based design of embedded systems: MATLAB Simulink/Stateflow (MSS). It results that our solutions have better time performance and a more stable implementation, because the adaptation of the period length is not based on the error minimization of differential equations. In some scenarios, where is more important the time performances than the precision of the simulation, our solutions can be preferred. More important is that in our solutions is possible to specify the desired precision of the simulation in regions of interest. Unfortunately, to the best of our knowledge we did not find benchmark sets specific for dynamic sampling algorithms. Furthermore is not possible to execute a customized sampling algorithm in the Simulink/Stateflow environment, but only in the MATLAB environment. Therefore, it was not possible to test the algorithms with Simulink/Stateflow models. In our approaches the setting of the period length and its adaptation mechanisms can be done directly specifying desired simulation situations, with respect to the algorithms in MSS. For instance a system validation can be done specifying the desired precision of the simulation for the error states or events of the system. These parameters could be directly derived from system requirements.

A problem to be addressed in the final implementations is the set of adequate periods at run-time in a network of components, the lack of benchmark sets and so the difficulties to evaluate them. Industrial case studies need to be performed to prove our approaches and define eventually better heuristics. It would also be advantageous to execute these use cases not only in MATLAB environment but also in the AutoFOCUS 3 tool. Therefore, the realization of an implementation of our dynamic sampling algorithms in AutoFOCUS 3 will be an fundamental topic. It will be also interesting to use the HyCOMP verification process to determine acceptance values for the sampling based on gradients. For instance by testing systematically maximal intervals of values for each variable, in order to find automatically the most convenient acceptance value. Other automatic or assisted ways to guide the determination of parameters of the simulation could be studied. In this way, our algorithms can use information about the behaviour of continuous variables in an infinite time interval using formal verification. Meanwhile, existing approaches as for instance the sampling in the MSS environment, usually check the variables behaviour with a finite time horizon.

References

- [Alb96] P. Albrecht. The RungeKutta Theory in a Nutshell. *SIAM Journal on Numerical Analysis*, 33(5):1712–1735, 1996.
- [BDD⁺92] Manfred Broy, Frank Dederich, Claus Dendorfer, Max Fuchs, Thomas Gritzner, and Rainer Weber. The Design of Distributed Systems - An Introduction to FOCUS. Technical Report TUM-I9202, Technische Universität München, 1992.
- [BS89] P. Bogacki and L.F. Shampine. A 3(2) pair of Runge - Kutta formulas. *Applied Mathematics Letters*, 2(4):321 – 325, 1989.
- [Cam13] Alarico Campetelli. Dynamic Sampling for FOCUS Hybrid Components. In *Proceedings of the 3rd International Conference on Circuits, System and Simulation (ICSSS'13)*, volume 3(5), pages 402–406. International Journal of Modeling and Optimization, 2013.
- [CK90] J. R. Cash and Alan H. Karp. A Variable Order Runge-Kutta Method for Initial Value Problems with Rapidly Varying Right-hand Sides. *ACM Trans. Math. Softw.*, 16(3):201–222, September 1990.
- [DP80] J.R. Dormand and P.J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19 – 26, 1980.
- [Feh70] E. Fehlberg. Klassische Runge-Kutta-Formeln vierter und niedrigerer Ordnung mit Schrittweiten-Kontrolle und ihre Anwendung auf Wärmeleitungsprobleme. *Computing*, 6(1-2):61–71, 1970.
- [Hen96] T. A. Henzinger. The theory of Hybrid Automata. In *Proceedings of the 11th Annual IEEE Symposium on Logic in Computer Science, LICS '96*, pages 278–, Washington, DC, USA, 1996. IEEE Computer Society.
- [HF11] Florian Hölzl and Martin Feilkas. AutoFOCUS 3 - A Scientific Tool Prototype for Model-Based Development of Component-Based, Reactive, Distributed Systems. In *Model-Based Engineering of Embedded Real-Time Systems*, volume 6100 of *Lecture Notes in Computer Science*, pages 317–322. Springer Berlin / Heidelberg, 2011.
- [HNW93] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I (2nd Revised. Ed.): Nonstiff Problems*. Springer-Verlag New York, Inc., New York, NY, USA, 1993.
- [OWN96] Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab. *Signals & Systems (2nd Ed.)*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA, 1996.
- [PTVF92] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in C (2nd Ed.): The Art of Scientific Computing*. Cambridge University Press, New York, NY, USA, 1992.
- [SBB⁺02] Josef Stoer, Roland Bulirsch, Richard H. Bartels, Walter Gautschi, and Christoph Witzgall. *Introduction to numerical analysis*. Texts in applied mathematics. Springer, New York, 2002.
- [Zwi97] Daniel Zwillinger. *Handbook of Differential Equations, Third Edition*. Academic Press, November 1997.